Project's High-Level Design

on

Dockerized Marketing Planner Agent

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Names & Enrolment Numbers:

Sr No	Student Name	Enrolment
1	Akhilesh Tiwari	EN22CS301085
2	Aditi Gaikwad	EN22CS301040
3	Alfez Khan	EN22CS301102
4	Akshat Sakalye	EN22CS301093
5	Aanchal Chaturvedi	EN22CS301014
6	Aditya Patidar	EN22CS301060

Group Name: Group 06D1

Project Number: DO-06

Industry Mentor Name: Aashruti Shah (Program Director, Datagami)

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-26

Table of Contents

1. Introduction
    a. Scope of the document
    b. Intended Audience
    c. System overview
2. System Design
    a. Application Design
    b. Process Flow
    c. Information Flow
    d. Technology Stack & Engineering Details
    e. Key Design Considerations & Fallbacks
3. Data Design
    a. Data Model
    b. Data Access Mechanism
4. Interfaces (API Catalogue)
5. State and Session Management
6. Non-functional Requirement
    a. Security Aspects
    b. Performance Aspects
7. HLD vs LLD Comparison (Architectural evolution)

1 Introduction

1.1 Scope of the Document

This document outlines the high-level architecture, infrastructure orchestration, and data flow for the Marketing Planner Agent. It defines the system components, their interactions within a Kubernetes environment, and the technological stack chosen to fulfill the project requirements.

1.2 Intended Audience

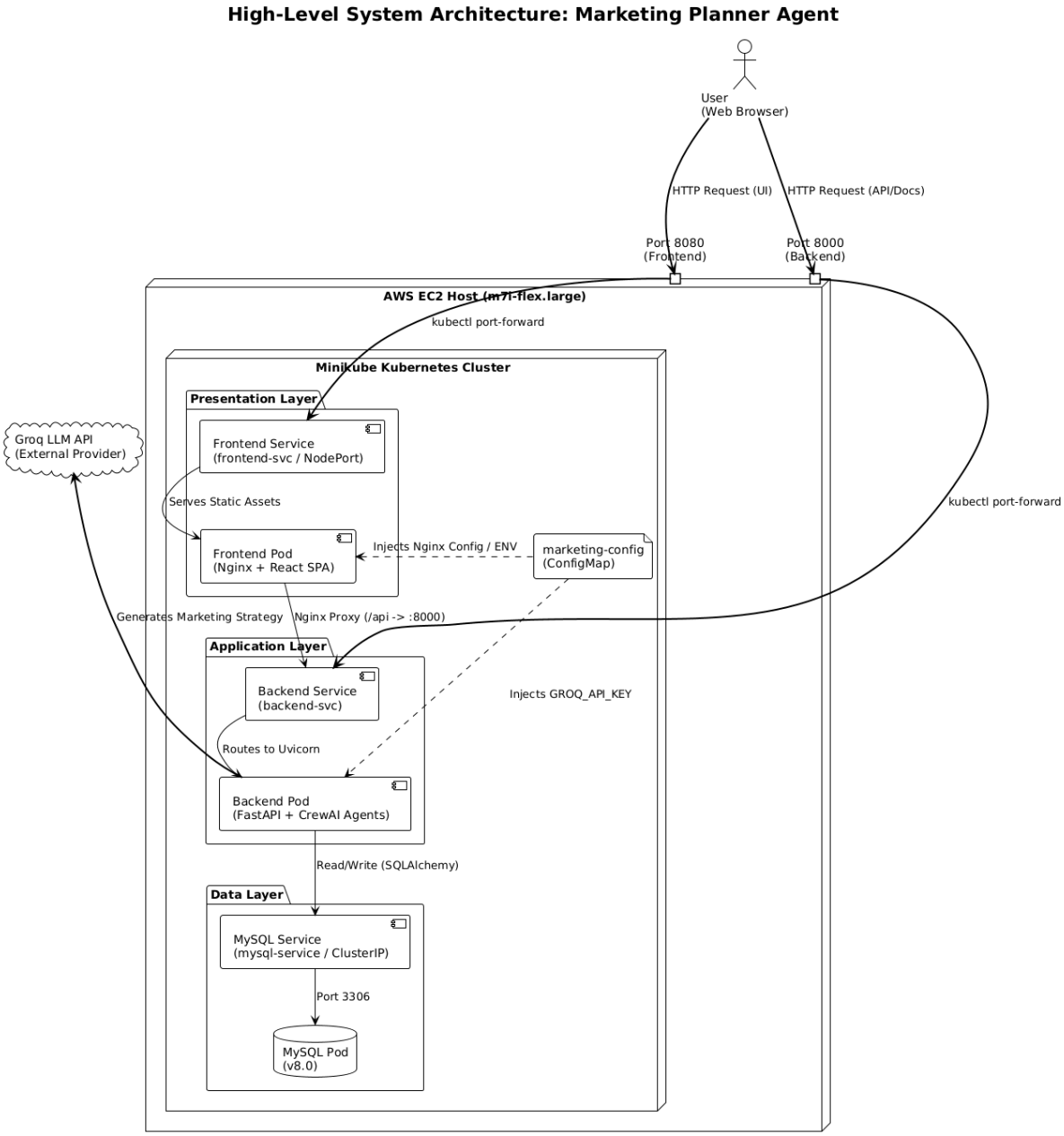
This document is intended for project evaluators, technical mentors, and the core engineering team to serve as the architectural blueprint.

1.3 System Overview

The Marketing Planner Agent is an AI-driven, full-stack application designed to automate and strategize marketing campaigns using Agentic AI. The system utilizes a decoupled architecture, featuring a highly optimized Single Page Application (SPA) frontend and a heavy, AI-capable backend. The entire ecosystem is containerized via Docker and orchestrated using a local Kubernetes cluster (Minikube) on AWS EC2, ensuring high availability, self-healing, and environmental portability.

2 System Design

2.1 Application Design



2.1 Application Design

The application follows a classic 3-Tier Microservices Architecture deployed within a Kubernetes cluster:

1. **Presentation Tier:** A React/Vite frontend served by Nginx.

2. **Application Tier:** A FastAPI Python backend integrating CrewAI for agentic workflows.
3. **Data Tier:** A containerized relational database.

2.2 Process Flow

1. The user accesses the web application via the EC2 Public IP on the exposed NodePort.
2. The browser downloads the static React assets.
3. User actions (e.g., Registration) trigger API calls to the relative path /api/....
4. The Nginx reverse proxy intercepts these calls and bridges them to the internal backend service.
5. The backend validates the request, updates the database, and (if applicable) initializes the AI agents to generate marketing content.

2.3 Information Flow

- **External to Cluster:** Client Browser → AWS Security Group → EC2 Host (Port 8080/8000) → K8s NodePort Service.
- **Intra-Cluster:** Frontend Service → Frontend Pod (Nginx).
    - Frontend Pod (Nginx Reverse Proxy) → Backend Service (Port 8000).
    - Backend Pod → MySQL Service (Port 3306).

2.4 Technology Stack & Engineering Details

- **Frontend:** React, Vite, Nginx (Alpine Linux, image size ~24 MB).
- **Backend:** Python 3.10, FastAPI, Uvicorn, CrewAI, Groq LLM API (image size ~2.96 GB).
- **Database:** MySQL 8.0.
- **DevOps & Infrastructure:** Docker, Kubernetes (Minikube), AWS EC2 (m7i-flex.large, 30GB EBS).

2.5 Key Design Considerations & Fallbacks

- **Environment Agnosticism:** To avoid tight coupling between the frontend build and the infrastructure IP, an Nginx reverse proxy was configured. The proxy_pass

directive handles dynamic URI routing (stripping trailing slashes to prevent 404 errors), allowing the same Docker image to be used across different environments.

- **Self-Healing Infrastructure:** By deploying via Kubernetes Deployments rather than standalone Docker containers, the system automatically recovers from application-level crashes (e.g., recovering from CrashLoopBackOff when the database connection is lost).

3 Data Design

3.1 Data Model

The system utilizes a relational model. Primary entities include:

- **Users:** Stores authentication credentials, email, and profile metadata.
- **Marketing Campaigns/Plans:** Stores the AI-generated strategies, linked to the User via foreign keys.

3.2 Data Access Mechanism

The backend utilizes **SQLAlchemy ORM** (Object-Relational Mapping) via pymysql to interact with the database. This prevents SQL injection and abstracts the database dialect from the core business logic.

4 Interfaces (API Catalogue)

The backend exposes a RESTful API. Key interfaces include:

- **Authentication:** POST /api/v1/auth/register - User onboarding.
    - POST /api/v1/auth/login - Generates access tokens.
- **Agent Operations:**
    - POST /api/v1/campaigns/generate - Triggers the CrewAI workflow.

5 State and Session Management

The backend is strictly **stateless** to conform to cloud-native 12-factor application principles.

- Session management is handled via **JWT (JSON Web Tokens)**.
- Upon successful login, the frontend stores the JWT in local storage and attaches it to the Authorization: Bearer header for all subsequent requests.

6 Non-functional Requirements

6.1 Security Aspects

- **Secrets Management:** API keys (e.g., GROQ_API_KEY) and database credentials are not baked into the Docker images. They are dynamically injected at runtime using Kubernetes ConfigMaps and Secrets.
- **Network Isolation:** The database pod does not have an external IP or NodePort. It is only accessible internally by the backend pod via the mysql-service ClusterIP.

6.2 Performance Aspects

- **Compute Allocation:** Deployed on an AWS m7i-flex.large instance to provide sufficient RAM (8GB) for the heavy Python/AI backend layer extraction and execution.
- **Frontend Delivery:** The frontend utilizes a multi-stage Docker build, resulting in a lightweight Nginx container that serves pre-compiled, minified static assets instantly.

7 HLD vs LLD Comparison (Architectural evolution)

While this High-Level Design focuses on the macro-architecture—specifically the orchestration of containers, network tunneling, and cloud infrastructure logic—the subsequent Low-Level Design (LLD) will detail the micro-architecture. The LLD will define the specific CrewAI agent assignments, the React component tree, state management hooks, and the internal FastAPI routing functions.